PRACTICAL NO. 4

DESIGN AND DEVELOP AN ANDROID APPLICATION

DEMONSTRATING SEND UPSTREAM MESSAGES

Android → Server Communication

1. AIM

To design and develop an Android application demonstrating Send Upstream Messages, where a message entered on the Android emulator is sent from the Android application to a server.

2. OBJECTIVE

To understand and implement the concept of upstream communication by creating an Android application with a message input field and Send Message button, and sending the entered message to a server using an HTTP POST request.

3. MOTIVE OF THE PRACTICAL — IN SIMPLE LANGUAGE

In simple words, this practical teaches us how an Android application can send information to a server. The user types a message in the application and presses the Send Message button. The Android application then sends that message to the server.

```
Downstream communication : Server → Android App
Upstream communication   : Android App → Server
```

In this practical, we demonstrate the second case: Android App → Server. The server is created using Python Flask and listens for the message sent by the Android emulator.

4. SOFTWARE / HARDWARE REQUIREMENTS

- Android Studio
- Java
- Android Emulator
- Python 3
- Flask Python library
- OkHttp library
- Internet/network access where required
- Firebase project and google-services.json (if your laboratory project requires Firebase configuration)

5. THEORY

5.1 Upstream Messaging

Upstream messaging refers to sending information from a client device/application toward a server. In this practical, the Android application acts as the client and the Python Flask application acts as the server.

5.2 Working Principle

```
User enters message
        ↓
Send Message button
        ↓
Android creates JSON data
        ↓
HTTP POST request using OkHttp
        ↓
Flask server receives request
        ↓
Server prints received message
        ↓
Android displays success message
```

6. PROCEDURE

Step 1: Create the Android Studio Project

1. Open Android Studio.
2. Select New Project.
3. Select Empty Views Activity.
4. Enter project name: UpstreamMessaging.
5. Select Java as the programming language.
6. Use package name: com.example.upstreammessaging.
7. Click Finish and wait for Gradle synchronization.

Step 2: Firebase Configuration

If the practical requires Firebase configuration, create/open the Firebase project and add the Android application using the exact package name. Download the generated google-services.json file.

8. Download google-services.json from Firebase.
9. Copy it into the app folder of the Android Studio project.
10. The file should be located at: UpstreamMessaging/app/google-services.json.
11. Do not rename the file.

```
UpstreamMessaging
└── app
    └── google-services.json
```

Step 3: Project-level build.gradle.kts

Open the project-level build.gradle.kts and use the following plugin configuration:

```
plugins {
    alias(libs.plugins.android.application) apply false
    id("com.google.gms.google-services") version "4.5.0" apply false
}
```

Step 4: Module-level build.gradle.kts

Open Gradle Scripts → build.gradle.kts (Module :app). Add the Google Services plugin, Firebase Messaging dependency, and OkHttp dependency.

```
plugins {
    alias(libs.plugins.android.application)
    id("com.google.gms.google-services")
}

android {
    namespace = "com.example.upstreammessaging"

    compileSdk = 36
```

```
        defaultConfig {
            applicationId = "com.example.upstreammessaging"
            minSdk = 24
            targetSdk = 36
            versionCode = 1
            versionName = "1.0"
        }

        compileOptions {
            sourceCompatibility = JavaVersion.VERSION_11
            targetCompatibility = JavaVersion.VERSION_11
        }
    }

    dependencies {

        implementation(platform("com.google.firebase:firebase-bom:34.18.0"))

        implementation("com.google.firebase:firebase-messaging")
        implementation("com.squareup.okhttp3:okhttp:4.12.0")

        implementation(libs.activity.ktx)
        implementation(libs.appcompat)
        implementation(libs.constraintlayout)
        implementation(libs.material)
    }
```

Click Sync Now after making the changes.

Step 5: Create the User Interface — activity_main.xml

Open app → res → layout → activity_main.xml. Delete the existing code and paste the following:

```
    <?xml version="1.0" encoding="utf-8"?>

    <androidx.constraintlayout.widget.ConstraintLayout
        xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        xmlns:tools="http://schemas.android.com/tools"
        android:id="@+id/main"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:context=".MainActivity">

        <TextView
            android:id="@+id/titleText"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Send Upstream Message"
            android:textSize="24sp"
            android:textStyle="bold"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            android:layout_marginTop="100dp" />

        <EditText
            android:id="@+id/messageInput"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:hint="Enter your message"
            android:inputType="text"
            app:layout_constraintTop_toBottomOf="@id/titleText"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            android:layout_marginStart="40dp"
            android:layout_marginEnd="40dp"
            android:layout_marginTop="30dp" />
```

```
    <Button
        android:id="@+id/sendButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Send Message"
        app:layout_constraintTop_toBottomOf="@id/messageInput"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="20dp" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 6: Update AndroidManifest.xml

Open app → manifests → AndroidManifest.xml. Internet permission is required because the Android application sends an HTTP request to the server. Cleartext traffic is enabled because the local demonstration server uses HTTP rather than HTTPS.

```
<?xml version="1.0" encoding="utf-8"?>

<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.UpstreamMessaging"
        android:usesCleartextTraffic="true">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>

                <action android:name="android.intent.action.MAIN" />

                <category
                    android:name="android.intent.category.LAUNCHER" />

            </intent-filter>

        </activity>

    </application>

</manifest>
```

Step 7: Update MainActivity.java

Open app → java → com.example.upstreammessaging → MainActivity.java. Delete the existing code and paste the following:

```
package com.example.upstreammessaging;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import java.io.IOException;

import okhttp3.Call;
import okhttp3.Callback;
```

```
import okhttp3.MediaType;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.RequestBody;
import okhttp3.Response;

public class MainActivity extends AppCompatActivity {

    EditText messageInput;
    Button sendButton;

    OkHttpClient client = new OkHttpClient();

    private static final String SERVER_URL =
            "http://10.0.2.2:5000/message";

    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        messageInput = findViewById(R.id.messageInput);
        sendButton = findViewById(R.id.sendButton);

        sendButton.setOnClickListener(v -> {

            String message =
                    messageInput.getText().toString().trim();

            if (message.isEmpty()) {

                Toast.makeText(
                        MainActivity.this,
                        "Please enter a message",
                        Toast.LENGTH_SHORT
                ).show();

                return;
            }

            sendMessage(message);
        });
    }

    private void sendMessage(String message) {

        MediaType JSON =
                MediaType.get("application/json; charset=utf-8");

        String json =
                "{\"message\":\"" + message + "\"}";

        RequestBody body =
                RequestBody.create(json, JSON);

        Request request =
                new Request.Builder()
                        .url(SERVER_URL)
                        .post(body)
                        .build();

        client.newCall(request).enqueue(new Callback() {

            @Override
            public void onFailure(Call call, IOException e) {

                runOnUiThread(() -> {
```

```
                    Toast.makeText(
                            MainActivity.this,
                            "Failed to send message",
                            Toast.LENGTH_SHORT
                    ).show();
                });
            }

            @Override
            public void onResponse(
                    Call call,
                    Response response) throws IOException {

                runOnUiThread(() -> {

                    Toast.makeText(
                            MainActivity.this,
                            "Message sent successfully",
                            Toast.LENGTH_SHORT
                    ).show();

                    messageInput.setText("");
                });
            }
        });
    }
}
```

Step 8: Create the Python Server

The server receives the upstream message. The Android emulator communicates with the computer's local server through 10.0.2.2.

12. Open Command Prompt.
13. Type: notepad server.py
14. When Notepad opens, paste the server code given below.
15. Save the file as server.py in C:\Users\harsh (or another folder you can easily access).
16. Make sure the file is server.py, not server.py.txt.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/message", methods=["POST"])
def receive_message():

    data = request.get_json()

    message = data.get("message")

    print("================================")
    print("UPSTREAM MESSAGE RECEIVED")
    print("Message:", message)
    print("================================")

    return jsonify({
        "status": "success",
        "message": "Message received by server"
    })


app.run(host="0.0.0.0", port=5000)
```

Step 9: Install Flask

Open Command Prompt and run:

```
python -m pip install flask
```

If Flask is already installed, Command Prompt will show 'Requirement already satisfied'.

Step 10: Start the Server

17. Open Command Prompt in the folder where server.py is saved.
18. Run: python server.py
19. Keep the Command Prompt window open while testing the Android application.

Expected server output:

```
* Serving Flask app 'server'
* Debug mode: off
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.x.x:5000
```

Step 11: Understand the Server URL

The Android code uses http://10.0.2.2:5000/message. In the Android Studio emulator, 10.0.2.2 refers to the host computer's localhost. Therefore, the Android emulator can reach the Flask server running on the computer.

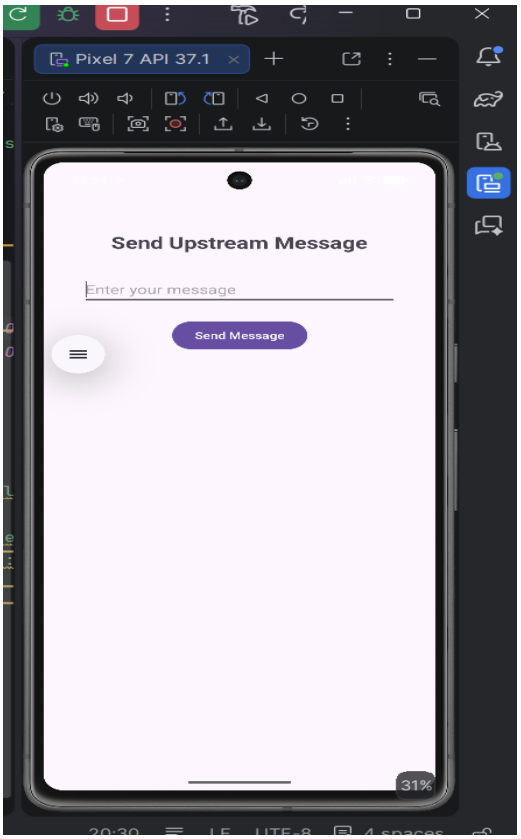
```
Android Emulator
      ↓
10.0.2.2:5000
      ↓
Computer / Flask Server
      ↓
/message
```

Step 12: Run the Android Application

20. Start the Android Emulator.
21. In Android Studio, click Run .
22. Wait for the application to open.
23. The title Send Upstream Message should appear.
24. Enter a message such as Hello from Android.
25. Press Send Message.

7. EXPECTED OUTPUT

7.1 Android Emulator — Initial Screen



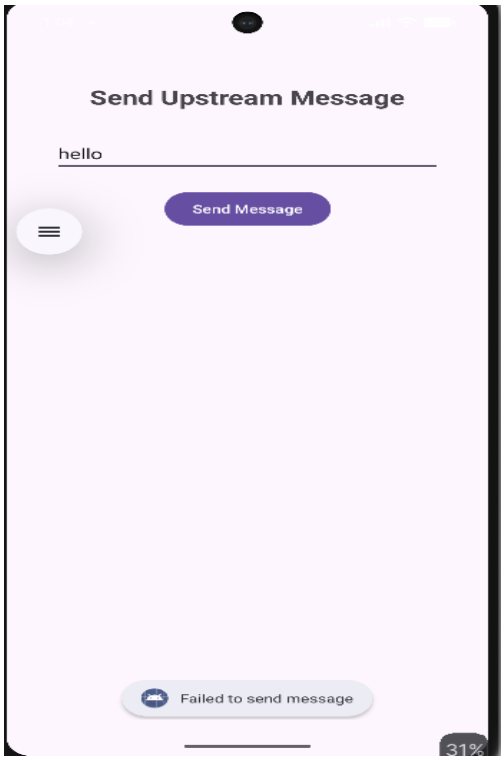
7.2 Entering the Message

```
+--------------------------------------+
|        Send Upstream Message         |
|                                      |
| [ Hello from Android              ]  |
|                                      |
|           [ Send Message ]            |
+--------------------------------------+
```

7.3 After Clicking Send Message

The Android application should display:

```
[ Toast ]  Message sent successfully
```



The message field is cleared after a successful response.

7.4 Flask Server Output

The Command Prompt running the Flask server should show:

```
================================
UPSTREAM MESSAGE RECEIVED
Message: hello msc part 2
================================
127.0.0.1 - - [10/Sep/2026 00:54:24] "POST /message HTTP/1.1" 200 -
================================
UPSTREAM MESSAGE RECEIVED
Message: Hello cutiesss....:) This is Harshita Ma'am
================================
127.0.0.1 - - [10/Sep/2026 01:01:29] "POST /message HTTP/1.1" 200 -
```

8. WORKING OF THE APPLICATION

26. The application starts and displays a text field and Send Message button.
27. The user enters a message.
28. The application checks whether the message is empty.
29. The message is placed inside JSON data.
30. OkHttp creates an HTTP POST request.
31. The request is sent to http://10.0.2.2:5000/message.
32. The Flask server receives the POST request.
33. The server extracts the message from the JSON object.
34. The server prints the message in the Command Prompt.
35. The server returns a JSON success response.

36. The Android application displays 'Message sent successfully' and clears the input field.

9. EXPLANATION OF IMPORTANT CODE

Code / Component	Purpose
EditText messageInput	Takes the message entered by the user.
Button sendButton	Allows the user to start sending the message.
OkHttpClient	Creates the HTTP client used for network communication.
SERVER_URL	Stores the address of the Flask server endpoint.
MediaType JSON	Specifies that the request body is JSON data.
RequestBody	Stores the JSON message that will be sent.
Request.Builder()	Builds the HTTP POST request.
enqueue()	Sends the request asynchronously so the UI is not blocked.
onFailure()	Runs when the request cannot reach the server.
onResponse()	Runs when a response is received from the server.
Toast	Displays the result to the user.
Flask @app.route	Defines the server endpoint that receives the POST request.
request.get_json()	Reads the JSON data sent by Android.

10. FILES USED

File	Purpose
activity_main.xml	Designs the Android user interface.
MainActivity.java	Reads the message and sends it to the server.
AndroidManifest.xml	Adds Internet permission and allows local HTTP traffic.
build.gradle.kts (Module :app)	Adds Firebase and OkHttp dependencies.
google-services.json	Firebase configuration file downloaded for the Android project.
server.py	Python Flask server that receives and prints the upstream message.

11. COMMON ERRORS AND SOLUTIONS

Problem	Solution
Failed to send message	Make sure python server.py is still running. Also verify that the Android Emulator is being used and SERVER_URL is http://10.0.2.2:5000/message.
Cleartext HTTP traffic not permitted	Make sure android:usesCleartextTraffic="true" is inside the <application> tag.
Network permission problem	Make sure <uses-permission android:name="android.permission.INTERNET" /> is present.
server.py not found	Use cd to move to the folder containing server.py, or create the file with notepad server.py in the current folder.
Flask not installed	Run python -m pip install flask.
Android project cannot resolve OkHttp	Check that implementation("com.squareup.okhttp3:okhttp:4.12.0") is inside dependencies and click Sync Now.
No server output	Check that the Flask window is running and that the Android app actually sends the request.

12. CONCLUSION

The Android application was successfully designed and developed to demonstrate upstream communication. A message entered by the user on the Android emulator was sent to a Python Flask server using an HTTP POST request. The received message was displayed in the server terminal, and the Android application displayed a success message.

13. VIVA QUESTIONS AND ANSWERS

What is upstream messaging?

It is communication from an Android/client application to a server.

What is the direction of upstream communication?

Android/device → server.

What is downstream communication?

Server → Android/device.

Why is OkHttp used?

To send the HTTP request from the Android application to the server.

Why is Internet permission required?

The application needs network access to communicate with the server.

What is JSON?

A lightweight text format used to represent structured data.

Why is POST used?

POST is used to send data to the server.

What is Flask?

A Python web framework used here to create the receiving server.

What does 10.0.2.2 mean in the Android emulator?

It refers to the host computer from the Android emulator.

What happens when the user clicks Send Message?

The app creates JSON data, sends an HTTP POST request, and displays the result.

Where can the received message be seen?

In the Command Prompt running the Flask server.

14. OBSERVATION / OUTPUT TO ATTACH IN JOURNAL

- Screenshot 1: Android Studio project with the completed code.
- Screenshot 2: Android emulator showing the Send Upstream Message screen.

- Screenshot 3: Android emulator after entering the message.
- Screenshot 4: Android emulator showing 'Message sent successfully'.
- Screenshot 5: Command Prompt showing the Flask server running.
- Screenshot 6: Command Prompt showing 'UPSTREAM MESSAGE RECEIVED' and the received message.

15. CONCLUSION

Thus, the concept of upstream messaging was demonstrated successfully by sending a message from an Android application to a server. The Android application used OkHttp to create and send an HTTP POST request, while the Flask server received and displayed the message.

16. NOTE ABOUT CURRENT FIREBASE FCM

The supplied implementation uses Firebase project configuration and Firebase Messaging dependency because the practical is based on the Firebase messaging topic. The actual working Android-to-server demonstration above uses HTTP POST with OkHttp and Flask. This is intentional: the older Android FCM upstream send API commonly used in legacy college practicals has been decommissioned by Firebase. Therefore, the HTTP client-to-server demonstration provides the working upstream communication used in this practical.